SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING PROGRAMME

CSA3717– SOFTWARE TESTING

EXPERIMENT 10

COMPREHENSIVE TEST PLAN – LIBRARY MANAGEMENT SYSTEM

Aim:

To Develop a comprehensive Test Plan document for a chosen application (such as a Library Management System), outlining various testing strategies, methodologies, and criteria to ensure thorough evaluation and validation of the system's functionality.

1. Introduction

The Library Management System is used by librarians and members to manage books, search the catalogue, issue and return books, reserve titles, calculate overdue fines and maintain member records. This test plan defines the activities required to evaluate the system before release.

2. Test Objectives

- Verify that the system satisfies documented functional and non-functional requirements.
- Identify defects before release and provide reproducible evidence.
- Check data accuracy, access control, usability and reliability.
- Confirm that fixes do not introduce regressions.

3. Test Scope

In-scope activities:

- Member registration and login
- Book catalogue search, filtering and availability display
- Book issue, return and renewal
- Reservation and cancellation
- Fine calculation and payment recording
- Librarian role and catalogue maintenance
- Reports and audit records

Out of scope: production data modification, live financial transactions, third-party services that are not available in the test environment, and features not included in the approved requirements.

4. Features to be Tested

Module / Feature	Testing Focus
Member Management	Registration, login, profile update, role permissions.
Book Catalogue	Search accuracy, filters, book details and availability.
Circulation	Issue, return, renewal, due dates and loan limits.
Reservations	Queue handling, availability notification and cancellation.
Fine Management	Overdue-day calculation, fine limits and payment status.
Reports	Issued books, overdue books, inventory and member activity.

5. Test Strategy and Methodology

Testing Level / Type	Planned Activities
Unit Testing	Developers test individual functions, validations and calculations.
Integration Testing	Verify communication between modules, databases and connected services.
System Testing	Execute end-to-end workflows against the complete application.
Regression Testing	Repeat selected tests after defect fixes or code changes.
Usability Testing	Check clarity of navigation, labels, messages and accessibility.
Security Testing	Check authentication, authorization, session handling and input validation.
Performance Testing	Measure response time and stability under expected concurrent usage.

6. Test Design Techniques

Technique	Application
Equivalence Partitioning	Divide inputs into valid and invalid groups and test representative values.
Boundary Value Analysis	Test minimum, maximum and values immediately below/above limits.
Decision Table Testing	Test combinations of conditions that determine an outcome.
State Transition Testing	Verify valid and invalid transitions between workflow states.
Use-Case Testing	Validate complete user journeys from start to completion.

7. Test Environment and Test Data

- Use a dedicated staging environment that is separate from production.
- Use synthetic accounts, records and transactions; do not expose real customer data.
- Record browser/device, operating system, application build and database version.
- Prepare valid, invalid, boundary and role-specific test data.
- Enable application logs and test evidence capture.

8. Test Execution Strategy

1. Review requirements and identify test conditions.
2. Prepare member accounts, book records, due dates and role-based test data.
3. Execute module-level tests, followed by integrated issue/return/reservation workflows.
4. Run boundary tests for loan limits, due dates and fine calculations.
5. Log defects, retest corrections and perform regression testing.
6. Prepare a test summary with pass rate, open defects and release risks.

9. Entry and Exit Criteria

Entry Criteria	**Exit / Completion Criteria**
Requirements are reviewed and approved; a stable test build and staging database are available; test accounts and test data are prepared.	All critical test cases pass; no open critical or high-severity defects remain without formal approval; planned regression tests pass; test results and known limitations are documented.

10. Defect Management

Each defect must include a unique ID, title, environment, preconditions, reproducible steps, expected result, actual result, severity, priority, evidence and current status. Defects move through New → Assigned → In Progress → Fixed → Retesting → Closed. Failed retests are reopened.

11. Risks and Mitigation

Risk	Mitigation
Incorrect fine calculation	Test date boundaries and independently verify fine calculations.
Duplicate book issue	Use transaction locking and test simultaneous issue attempts.
Loss of test data	Take controlled backups and use resettable synthetic data.
Role permission gaps	Test every key function using member and librarian roles.

12. Deliverables

- Approved test plan and requirement coverage list.
- Test scenarios, test cases and test data.
- Execution log with Pass / Fail / Blocked status.
- Defect reports with supporting evidence.
- Test summary report and release recommendation.

13. Conclusion

This test plan defines a structured approach for evaluating the Library Management System. Applying the planned test levels, design techniques, execution controls and completion criteria helps improve product quality and reduces the risk of defects reaching users.